

PROTOTYPES, CLASSES

STRINGS

Why is function includes accessible? Where it's defined?

```
const str = 'Hello world';  
  
'Hello world'.includes('world'); // true
```

ARRAYS

Why is property length accessible? Where it's defined?

```
const arr = [1, 2, 3];  
  
for (let i = 0; i < arr.length; ++i) {  
  console.log(arr[i]);  
}
```

[[PROTOTYPE]] OBJECT ACCESSIBLE BY `__PROTO__`

*`__proto__` is a reference to the object
prototype*

```
const arr = [1, 2, 3];  
  
arr.__proto__; // { at: f, concat: f, find: f, ..., length, ..  
[false, true].__proto__; // { at: f, concat: f, find: f, ..
```

[[PROTOTYPE]] OBJECT ACCESSIBLE BY __PROTO__

```
const animal = {  
  eats: true,  
};  
  
const rabbit = {  
  jumps: true,  
  __proto__: animal, // only for educational purposes  
};  
  
console.log(animal.eats, animal.jumps, rabbit.eats, rabbit.jumps);
```

[[PROTOTYPE]] OBJECT ACCESSIBLE BY __PROTO__

- __proto__ could reference to an object or null, other types ignored
- Circular references are not allowed
- An object could have only one prototype (one __proto__ reference)

DON'T USE `__PROTO__` DIRECTLY, USE METHODS INSTEAD

- To get (read) prototype of an object use: `Object.getPrototypeOf(obj)`
- To set a prototype to an object use: `Object.setPrototypeOf(obj, somePrototype);`

```
const animal = {
  eats: true,
};

const rabbit = {
  jumps: true,
};

Object.getPrototypeOf(rabbit); // { hasOwnProperty: f, ... } -> some other obj
Object.setPrototypeOf(rabbit, animal);
Object.getPrototypeOf(rabbit); // { eats: true }
```

ABOUT THIS

```
let animal = {
  walk: function () {
    if (!this.isSleeping) {
      console.log(`I walk`);
    }
  },
  sleep: function () {
    this.isSleeping = true;
  },
};

let rabbit = {
  name: 'White Rabbit',
};

Object.setPrototypeOf(rabbit, animal);

// controls isSleeping inside rabbit object
rabbit.sleep();

console.log(rabbit.isSleeping); // true
console.log(animal.isSleeping); // undefined
```


ACCESS TO OBJECT OWN PROPERTIES: OBJECT.KEYS(OBJ)

Object.keys - returns an array of strings (names of object own keys)

```
let animal = {  
  sleep: function () {  
    console.log(`${this.name} is sleeping`);  
  },  
};  
  
let rabbit = {  
  name: 'White Rabbit',  
};  
  
Object.setPrototypeOf(rabbit, animal);  
  
console.log(Object.keys(rabbit));  
console.log(Object.keys(animal));
```

ACCESS TO OBJECT OWN PROPERTIES: LOOP FOR..IN

Loop for..in - iterates through object own properties and inherited (if a property has enumerable: true)

```
let animal = {
  sleep: function () {
    console.log(`${this.name} is sleeping`);
  },
};

let rabbit = {
  name: 'White Rabbit',
};

Object.setPrototypeOf(rabbit, animal);

for (let prop in rabbit) {
  console.log(prop); // 'name', 'sleep'
}
```

ACCESS TO OBJECT OWN PROPERTIES: FILTER OWN PROPERTIES

An object has a property `hasOwnProperty` which returns boolean value if an object has a property passed to a `hasOwnProperty` function as a parameter

```
let animal = {
  sleep: function () {
    console.log(`${this.name} is sleeping`);
  },
};

let rabbit = {
  name: 'White Rabbit',
};

Object.setPrototypeOf(rabbit, animal);

for (const prop in rabbit) {
  const isOwn = rabbit.hasOwnProperty(prop);

  if (isOwn) {
    console.log(`${prop} is own property`); // name is own property
  } else {
    console.log(`${prop} inherited property`); // sleep is inherited property
  }
}
```

OBJECT PROPERTY DESCRIPTORS

```
const person = {  
  name: 'Ivan',  
};  
  
console.log(person.name); // 'Ivan'  
  
const descriptor = Object.getOwnPropertyDescriptor(person, 'name');  
  
console.log(descriptor); // { "value": "Ivan", "writable": true, "enumerable": true, "co
```

OBJECT PROPERTY DESCRIPTORS

- writable - if true, then a property is accessible to be changed, otherwise its readonly
- enumerable - if true, then a property is accessible in loops (i.e. for..in), otherwise loops are ignoring it
- configurable - if true, then a property could be deleted and attributes could be changed, otherwise not

Errors are displayed in "use strict;" mode

OBJECT READONLY PROPERTY

Changing an existing property descriptor

```
'use strict';

const person = {
  name: 'Ivan',
};

Object.defineProperty(person, 'name', {
  writable: false,
});

person.name = 'Maria'; // Uncaught TypeError: Cannot assign to read only property 'name'
```

OBJECT READONLY PROPERTY

Adding a new property with descriptor

```
'use strict';

const person = {};

Object.defineProperty(person, 'name', {
  value: 'Ivan'
  writable: false,
});

person.name = 'Maria'; // Uncaught TypeError: Cannot assign to read only property 'name'
```

OBJECT PROPERTY DESCRIPTORS

More info https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/defineProperty

OBJECT GETTERS AND SETTERS

An object properties could be defined as

- data properties
- accessor properties

```
const person = {
  firstname: 'Ivan',
  lastname: 'Ivanov',

  // when person.fullname is being called
  get fullname() {
    return `${this.firstname} ${this.lastname}`;
  },

  set fullname(value) {
    // value: 'Maria Abramovich'
    const values = value.split(' '); // ['Maria', 'Abramovich']

    this.firstname = values[0];
    this.lastname = values[1];

    // All three lines could be replaced by one line
    // [this.firstname, this.lastname] = value.split(' ');
    // See https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/De
  },
};
```

PROTOTYPES ON CUSTOM FUNCTIONS

CREATING AN OBJECT

Simple approach

```
const person = { name: 'Ivan', age: 18 };
```

Generic approach to use a function to create new instances of a Person prototype

```
function Person(name, age) {  
  this.name = name;  
  this.age = age;  
  this.sayHi = () => {  
    console.log(this.name);  
  };  
}  
const person = new Person('Ivan', 18);  
person.sayHi(); // 'Ivan'
```

"PROTOTYPE" PROPERTY ON FUNCTION

- Each normal function (not arrow) by default has a property "prototype"
- By default "prototype" referenced to an object with a property "constructor" referenced to the function constructor

```
function Person(name) {  
  this.name = name;  
}  
  
// Person.prototype = { constructor: Person }  
  
console.log(Person.prototype.constructor == Person); // true  
  
const person = new Person('Ivan');  
  
console.log(person.constructor == Person); // true  
  
const maria = new person.constructor('Maria');  
  
console.log(maria); // { name: "Maria" }  
console.log(person.constructor == maria.constructor); // true
```

"CONSTRUCTOR" PROPERTY

- JavaScript doesn't guarantee to keep the reference to the valid object inside "constructor" property
- By default it's function, but we could change it

```
function Person(name) {  
  this.name = name;  
}  
  
Person.prototype = {  
  sayHi: function () {  
    console.log('hi', this.name);  
  },  
};  
  
const person = new Person('Ivan');  
  
console.log(person);  
console.log(person.constructor); // undefined
```

AVOID REPLACING FUNCTION PROTOTYPE COMPLETELY

Don't replace a function prototype, add properties to it

```
function Person(name) {  
  this.name = name;  
}  
  
Object.defineProperty(Person.prototype, 'age', {  
  value: 18, // default value  
  writable: true,  
  enumerable: true,  
  configurable: true,  
});  
  
const person = new Person('Ivan', 18);  
  
console.log(person); // { name: 'Ivan' }  
console.log(person.age); // 18  
  
for (const prop in person) {  
  console.log(prop, person.hasOwnProperty(prop)); // name true, age false  
}
```

DEFINE FUNCTIONS IN PROTOTYPES, NOT IN CONSTRUCTOR FUNCTIONS

Problem: sayHi is an inner property of an instance created by a function Person. As many object instances you're creating, the more copied sayHi functions are being created too

```
function Person(name) {  
  this.name = name;  
  this.sayHi = function () {  
    console.log(`Hi ${this.name}`);  
  };  
}  
  
const person = new Person('Ivan');  
const maria = new Person('Maria');  
const susy = new Person('Susy');  
  
person.sayHi(); // 'Hi Ivan'  
maria.sayHi(); // 'Hi Maria'  
susy.sayHi(); // 'Hi Susy'
```

DEFINE FUNCTIONS IN PROTOTYPES, NOT IN CONSTRUCTOR FUNCTIONS

Solution: define functions in prototypes to save memory and avoid copying of functions

```
function Person(name) {  
  this.name = name;  
}  
  
Person.prototype.sayHi = function () {  
  console.log(`Hi ${this.name}`);  
};  
  
const person = new Person('Ivan');  
const maria = new Person('Maria');  
const susy = new Person('Susy');  
  
person.sayHi(); // 'Hi Ivan'  
maria.sayHi(); // 'Hi Maria'  
susy.sayHi(); // 'Hi Susy'
```


INHERITANCE

OBJECT.CREATE() METHOD

Object.create() creates a new object, using an existing object as the prototype of the newly created object.

```
const person = {
  isHuman: false,
  printIntroduction: function () {
    console.log(`My name is ${this.name}. Am I human? ${this.isHuman}`);
  },
};

const me = Object.create(person); // person should be the prototype

me.name = 'Matthew'; // "name" is a property set on "me", but
me.isHuman = true; // Inherited properties can be overwritten

me.printIntroduction();
// Expected output: "My name is Matthew. Am I human? true"
```

INHERITANCE

```
function User(name) {  
  this.name = name;  
}  
  
User.prototype.sayHi = function () {  
  console.log('I am a user', this.name);  
};  
  
function Admin(name) {  
  User.call(this, name); // call parent constructor  
}  
  
Admin.prototype = Object.create(User.prototype);  
Admin.prototype.constructor = Admin;
```

CLASSES

CLASSES

In many other languages, classes, or constructors, are clearly distinguished from objects, or instances. In JavaScript, classes are mainly an abstraction over the existing prototypical inheritance mechanism — all patterns are convertible to prototype-based inheritance. Classes themselves are normal JavaScript values as well, and have their own prototype chains.

CLASSES: SYNTAX

- Define with keyword class
- Provide a name for the class
- Describe the body of the class
- Create an instance of a class with keyword new
- Use a class instance

```
class Person {  
  name = 'Ivan';  
}  
  
const person = new Person();  
person.name; // 'Ivan'
```

CLASSES: METHODS DEFINED WITH =

A class field is a property defined directly in a class body

Class fields are accessible inside class body and outside of it

```
class Person {  
  name = 'Ivan'; // class field called 'name'  
}  
  
const person = new Person();  
person.name; // access to the class field called 'name', will print 'Ivan'  
person.name = 'Maria'; // access to the class field called 'name' and set a new value  
person.name; // 'Maria'
```

CLASSES: FIELDS

Class fields can store a function. A function defined in such way is accessible in the class instance, not in the class prototype

```
class Person {  
  name = 'Ivan'; // class field called 'name'  
  
  // class method defined with = is stored in the class instance, not in the class prototype  
  getAge = function () {  
    return 22;  
  };  
}  
  
const person = new Person();  
  
person.getAge(); // 22  
  
console.log(person); // { name: 'Ivan', getAge: f, [[Prototype]]: { constructor: class Person }}  
console.log(Person.prototype); // { constructor: class Person, [[Prototype]]: Object }
```


CLASSES: METHODS DEFINED WITHOUT =

A function defined in such way is accessible in the class prototype

CLASSES: CONSTRUCTOR

A constructor() is the default prototypal method that comes built-in with every JavaScript class.

It's optional to define constructor method inside a class body. However, if you do not create one, JavaScript will automatically add an empty one.

Only one **construct** method is allowed

```
class Person {  
  constructor(name) {  
    // this refers to the class instance  
    this.name = name;  
  }  
}  
  
const person = new Person('Ivan');  
console.log(person); // { name: 'Ivan' }
```

CLASSES: BEST PRACTICES SO FAR

Define and classes when you need to have a template for creating objects that:

- could have group some data inside an instance
- provide functions to work with such data

Class definition

- put methods in the class propotype, not in the class instance, to get benefits from propotypical inheritance

```
class Person {
  constructor(firstname, lastname) {
    this.firstname = firstname;
    this.lastname = lastname;
  }

  introduce() {
    return `Hi, my name is ${this.firstname} ${this.lastname}`;
  }
}

const ivan = new Person('Ivan', 'Martynov');
ivan.introduce(); // 'Hi, my name is Ivan Martynov'
```

CLASSES: ENCAPSULATION AND CLASS FIELDS

In software systems, encapsulation refers to the bundling of data with the mechanisms or methods that operate on the data. It may also refer to the limiting of direct access to some of that data, such as an object's components. Essentially, encapsulation prevents external code from being concerned with the internal workings of an object.

CLASSES: ENCAPSULATION AND CLASS FIELDS

Three types of class fields are:

- Public class fields
- Private class fields
- Static class fields

CLASSES: PUBLIC CLASS FIELDS

Accessible for read and write inside class body and outside of the class body (class instance)

```
class Person {  
  age = 20; // default value for public class field 'age'  
  
  constructor(name) {  
    this.name = name; // set value for public class field 'name' on object instance  
  }  
  
  introduce() {  
    return `Hi, my name is ${this.name} and I'm ${this.age} years old`;  
  }  
}  
  
const ivan = new Person('Ivan');  
ivan.introduce(); // access to public class methods  
  
ivan.name = 'Maria'; // access to public class field and set new value  
  
ivan.age; // access to public class field and get value
```

CLASSES: PRIVATE CLASS FIELDS

Accessible for read and write inside class body and NOT outside of the class body (class instance)

Use symbol # to define private class field

```
class Person {
  #age = 20; // default value for private class field '#age'
  #lastname; // default value for private class field '#lastname' is undefined

  constructor(firstname, lastname) {
    this.firstname = firstname; // set value for public class field 'name' on object instance
    this.#lastname = lastname; // set value for private class field 'name' on object instance
  }

  introduce() {
    return `Hi, my name is ${this.firstname} ${this.#lastname} and I'm ${
      this.#age
    } years old`;
  }
}

const ivan = new Person('Ivan', 'Martynov');
ivan.introduce(); // access to public class methods

ivan.age; // access to private class field will result as undefined
ivan.#age; // access to private class field will result with an error
```

CLASSES: STATIC CLASS FIELDS

Accessible for read and write from the class itself - those fields are not in the class instance or in the class prototype

Use keyword static to define static class fields

```
class Person {
  static age = 20; // default value for private class field '#age'

  constructor(name) {
    this.name = name; // set value for public class field 'name' on object instance created
  }

  introduce() {
    // static field access inside class body
    return `Hi, my name is ${this.name} and I'm ${Person.age} years old`;
  }

  static print() {
    console.log('Static method called');
  }
}

const ivan = new Person('Ivan');

Person.age; // access to the static class field
Person.print(); // access to the static class method
```


CLASSES: ENCAPSULATION BEST PRACTICES SO FAR

- Encapsulate data reasonably with private class fields
- Use getters/setters class methods to provide an access to the private class fields if needed
- Use static class fields to store some constants or "magic values"

CLASSES: ENCAPSULATION BEST PRACTICES SO FAR

Use getters/setters class methods to provide an access to the private class fields if needed

```
class Person {
  #name;

  constructor(name) {
    this.#name = name;
  }

  get name() {
    return this.#name;
  }

  set name(value) {
    // possibility to add some checks and prevent possible errors
    if (typeof value === 'string') {
      this.#name = value;
    } else {
      this.#name = '';
    }
  }

  sayHi() {
    console.log(`Hi, my name is ${this.#name}`);
  }
}
```

CLASSES: ENCAPSULATION BEST PRACTICES SO FAR

Use static class fields to store some constants or "magic values"

```
class Colors {
  static RED = '#ff0000';
  static GREEN = '#00ff00';
  static BLUE = '#0000ff';

  static ALL = [Colors.RED, Colors.GREEN, Colors.BLUE];

  static getColorName(colorCode) {
    return colorCode === Colors.RED
      ? 'red'
      : colorCode === Colors.GREEN
      ? 'green'
      : colorCode === Colors.BLUE
      ? 'blue'
      : 'unknown';
  }
}

Colors.GREEN; // '#00ff00'
Colors.ALL; // ['#ff0000', '#00ff00', '#0000ff']
Colors.getColorName(Colors.BLUE); // 'blue'
```

CLASSES: DERIVED CLASS

A derived class is a class that extends the public and static features of an existing class (parent class). It cannot access its parent class private fields.

- Defined with a keyword extends
- A base class is called a parent class
- A derived class is called a child class

```
class BaseClass {}  
  
class DerivedClass extends BaseClass {}
```

CLASSES: DERIVED CLASS EXAMPLE

```
class Person {}

class Student extends Person {}

const ivan = new Person();
const maria = new Student();

console.log(ivan); // { [[Prototype]]: { constructor: class Person, [[Prototype]]:
console.log(Person.prototype); // { constructor: class Person, [[Prototype]]:
console.log(maria); // { [[Prototype]]: { constructor: class Student, [[Prototype]]:
console.log(Student.prototype); // { constructor: class Student, [[Prototype]]:
```

CLASSES: DERIVED CLASS EXAMPLE

```
class Person {  
  name = 'common';  
}  
  
class Student extends Person {}  
  
const ivan = new Person();  
const maria = new Student();  
  
console.log(ivan); // { name: 'common' }  
console.log(maria); // { name: 'common' }
```

CLASSES: DERIVED CLASS EXAMPLE

A derived class field will override parent class field with the same name

```
class Person {  
  name = 'parent';  
}  
  
class Student extends Person {  
  name = 'student';  
}  
  
const ivan = new Person();  
const maria = new Student();  
  
console.log(ivan); // { name: 'parent' }  
console.log(maria); // { name: 'student' }
```

CLASSES: DERIVED CLASS SUPER KEYWORD

The super keyword searches a parent class or object literal for a specified static or prototypal property.

You can use the super keyword as a “function caller” or “property accessor.”

```
super(arg1, arg2, ...); // function caller
```

```
super.someField; // property accessor
```

```
super.someMethod(); // property accessor
```


CLASSES: SUPER AS A FUNCTION CALLER

```
class Person {
  constructor(name) {
    this.name = name;
  }
}

class Student extends Person {
  constructor(name, age) {
    super(name); // parent constructor call with arguments
    this.age = age;
  }
}

const ivan = new Person('Ivan');
const maria = new Student('Maria', 22);

console.log(ivan); // { name: 'Ivan' }
console.log(maria); // { name: 'Maria', age: 22 }
```

CLASSES: SUPER AS A FUNCTION CALLER

- Calling `super()` allows JavaScript to use the parent class's constructor to initialize this.
- It's required to call `super()` before using the keyword `this`. Otherwise, an error will be thrown.

```
class Person {
  constructor(name) {
    this.name = name;
  }
}

class Student extends Person {
  constructor(name, age) {
    super(name); // parent constructor call with arguments
    this.age = age;
  }
}

const ivan = new Person('Ivan');
const maria = new Student('Maria', 22);

console.log(ivan); // { name: 'Ivan' }
console.log(maria); // { name: 'Maria', age: 22 }
```



CLASSES: SUPER AS A PROPERTY ACCESSOR

```
class Person {
  #minAge = 18;

  constructor(name) {
    this.name = name;
  }

  get minAge() {
    return this.#minAge;
  }
}

class Student extends Person {
  constructor(name, age) {
    super(name);

    this.age = age;
  }

  isAgeValid() {
    return this.age >= super.minAge;
  }
}
```

CLASSES AND ARROW FUNCTIONS

CLASSES AND ARROW FUNCTIONS

- sayHi method is defined in the class instance, not in the prototype

```
class Person {  
  name = 'Ivan';  
  
  sayHi = () => {  
    console.log(this.name);  
  };  
}  
  
const person = new Person();  
  
person.sayHi(); // 'Ivan'
```

CLASSES AND ARROW FUNCTIONS

- sayHi method keeps a reference in this to the execution context when has been created

```
class Person {  
  name = 'Ivan';  
  
  sayHi = () => {  
    console.log(this.name);  
  };  
}  
  
const person = new Person();  
  
person.sayHi(); // 'Ivan'  
  
const { sayHi } = person;  
  
sayHi(); // 'Ivan' --> finds 'name' in the execution context where this is bir
```

CLASSES AND ARROW FUNCTIONS

- not possible to use as constructor function

```
class Person {  
  constructor = (name) => {  
    // error  
    this.name = name;  
  };  
}  
  
const person = new Person('Ivan');
```


USED MATERIALS

<https://www.freecodecamp.org/news/javascript-class-handbook/>

<https://www.freecodecamp.org/news/javascript-arrow-functions-in-depth/#heading-when-you-should-not-use-arrow-functions>

<https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes>

<https://prateeksurana.me/blog/how-javascript-classes-work-under-the-hood/#>

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions/Arrow_functions

https://www.reddit.com/r/learnjavascript/comments/1bk8600/arrow_